

# Industrial Blockchain Laboratory

## Project AeroTrack:

Securing Airplane Spare Parts Provenance using Ethereum  
Smart Contracts

### Lab Objectives

- Understand the critical role of blockchain in industrial dependability and supply chain security.
- Apply previously learned Solidity concepts (Structs, Mappings, Modifiers, Events) to a real-world engineering problem.
- Implement Role-Based Access Control (RBAC) to ensure only authorized entities can modify industrial data.
- Deploy, test, and debug a comprehensive smart contract using the Remix IDE.

Designed for Future Engineers

May 18, 2026

# Contents

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction to the Industrial Context</b>                    | <b>2</b>  |
| 1.1      | The Blockchain Solution . . . . .                                | 2         |
| 1.2      | What You Must Understand Before We Start . . . . .               | 2         |
| <b>2</b> | <b>Step 1: Architecting the Data Structure</b>                   | <b>3</b>  |
| 2.1      | The <code>Part</code> Struct . . . . .                           | 3         |
| 2.2      | The <code>parts</code> Mapping . . . . .                         | 3         |
| <b>3</b> | <b>Step 2: Role-Based Access Control (Modifiers)</b>             | <b>5</b>  |
| 3.1      | Using <code>modifier</code> . . . . .                            | 5         |
| 3.2      | Understanding the Logic . . . . .                                | 6         |
| <b>4</b> | <b>Step 3: Core Functions - Manufacturing &amp; Transferring</b> | <b>7</b>  |
| 4.1      | Manufacturing a Part . . . . .                                   | 7         |
| 4.2      | Transferring Ownership . . . . .                                 | 7         |
| <b>5</b> | <b>Step 4: Traceability and Maintenance Logging using Events</b> | <b>9</b>  |
| 5.1      | Defining Events . . . . .                                        | 9         |
| <b>6</b> | <b>The Complete Smart Contract</b>                               | <b>11</b> |
| <b>7</b> | <b>Step 5: Testing and Debugging in Remix IDE</b>                | <b>14</b> |
| 7.1      | Deployment . . . . .                                             | 14        |
| 7.2      | Simulating the Supply Chain . . . . .                            | 14        |
| <b>8</b> | <b>Conclusion and Future Enhancements</b>                        | <b>16</b> |
| 8.1      | Topics for Further Research . . . . .                            | 16        |

# 1 Introduction to the Industrial Context

In the aviation industry, dependability and safety are paramount. A single counterfeit or poorly maintained spare part can compromise the integrity of an entire aircraft, leading to catastrophic failures. Traditional supply chains rely on centralized databases and paper-based certificates (like the EASA Form 1 or FAA 8130-3), which are susceptible to forgery, loss, and data manipulation.

## 1.1 The Blockchain Solution

By leveraging Ethereum smart contracts, engineers can create an immutable, transparent, and decentralized ledger for tracking the lifecycle of an airplane part. From the moment it is manufactured, through various ownership transfers (airlines, maintenance facilities), to its final decommissioning, every action is cryptographically secured.

This laboratory merges core software engineering principles with industrial risk analysis. We will eliminate single points of failure in data tracking by utilizing decentralized consensus.

## 1.2 What You Must Understand Before We Start

To succeed in this lab, you need to recall the following concepts from previous sessions:

- **State Variables:** Data stored permanently on the blockchain.
- **Structs & Mappings:** Used to create complex data types and link them to unique identifiers (like a serial number).
- **msg.sender:** A global variable representing the Ethereum address calling the function. Essential for access control.
- **Modifiers & Require:** Used to enforce business logic (e.g., "Only the manufacturer can create a part").
- **Events:** Used to emit signals to the outside world, creating a traceable history without consuming expensive storage gas.

## 2 Step 1: Architecting the Data Structure

The first step in any engineering software project is defining the data architecture. We need to define what an "Airplane Part" is in the context of our smart contract.

### 2.1 The Part Struct

We will use a `struct` to group the attributes of a single spare part. Our part will need:

- A unique serial number.
- The name/description of the part (e.g., "Turbine Blade").
- The Ethereum address of the manufacturer who created it.
- The Ethereum address of the current owner (an airline or mechanic).
- A status indicating if it is newly built, installed, or retired.

### 2.2 The parts Mapping

To store millions of parts efficiently without iterating through arrays (which costs too much Gas), we will use a `mapping`. A mapping is like a hash table or dictionary. We will map a `uint256` (the serial number) directly to the `Part` struct.

#### Action: Write the Skeleton

Open Remix IDE, create a new file named `AeroTrack.sol`, and type the following code:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.19;
3
4 contract AeroTrack {
5
6     // Enum to track the physical state of the part
7     enum PartStatus { Manufactured, InTransit, Installed, Retired
8         }
9
10    // Struct representing an airplane spare part
11    struct Part {
12        uint256 serialNumber;
13        string partName;
14        address manufacturer;
15        address currentOwner;
```

```
15     PartStatus status;  
16     bool exists; // Security flag to check if part was  
           actually created  
17 }  
18  
19 // State variables  
20 address public regulatoryAuthority; // E.g., FAA or EASA  
21  
22 // Mapping a Serial Number to a Part struct  
23 mapping(uint256 => Part) public parts;  
24  
25 // Constructor runs once during deployment  
26 constructor() {  
27     regulatoryAuthority = msg.sender; // The deployer is the  
           authority  
28 }  
29 }
```

Listing 1: Defining the Contract and Data Structures

**Student Note:** Why do we use the `exists` boolean inside the struct? Because in Solidity, mappings return default values (zeros) if a key doesn't exist. The `exists` flag helps us verify that a serial number is legitimate and not just a default empty struct!

---

*Student Observations (Jot down your thoughts on struct optimization here):*

## 3 Step 2: Role-Based Access Control (Modifiers)

In industrial systems, security is not optional. We cannot allow a random mechanic to "mint" a new airplane part on the blockchain. Only certified manufacturers should have this right. Furthermore, only the *current owner* of a part should be able to transfer it to someone else.

### 3.1 Using modifier

A **modifier** is a piece of code that runs *before* a function executes. We use `require()` statements inside modifiers to check conditions. If the condition fails, the transaction reverts, saving gas and preventing illegal actions.

#### Action: Add Modifiers

Add the following modifiers to your AeroTrack contract, right below the constructor.

```
1 // --- MODIFIERS ---
2
3 // 1. Ensure the caller is the regulatory authority
4 modifier onlyAuthority() {
5     require(msg.sender == regulatoryAuthority, "Error: You
6         are not the Authority.");
7     _; // This underscore tells Solidity to execute the rest
8         of the function
9 }
10
11 // 2. Ensure a part actually exists before interacting with
12     it
13 modifier partExists(uint256 _serialNumber) {
14     require(parts[_serialNumber].exists == true, "Error: Part
15         does not exist in registry.");
16     _;
17 }
18
19 // 3. Ensure only the current owner of a specific part can
20     modify it
21 modifier onlyPartOwner(uint256 _serialNumber) {
22     require(parts[_serialNumber].currentOwner == msg.sender,
23         "Error: You do not own this part.");
24     _;
```

19

}

Listing 2: Adding Security Modifiers

### 3.2 Understanding the Logic

By applying `onlyPartOwner` to a transfer function later, we guarantee that if Airline A owns a Turbine Blade, Airline B cannot maliciously transfer it away from them. The blockchain verifies the cryptographic signature of `msg.sender` against the `currentOwner` address stored in the contract.

---

*Student Observations (Explain why the underscore `_;` is crucial in a modifier):*

## 4 Step 3: Core Functions - Manufacturing & Transferring

Now we will implement the business logic. We need functions to create new parts and to move them through the supply chain.

### 4.1 Manufacturing a Part

When a manufacturer creates a part, they input the serial number and name. The smart contract assigns them as both the `manufacturer` and the initial `currentOwner`.

### 4.2 Transferring Ownership

When the manufacturer sells the part to an airline, ownership changes. We must update the `currentOwner` and the `status` (e.g., to `InTransit`).

#### Action: Add Core Functions

Add these functions to your contract. Pay close attention to how the modifiers are attached to the function signatures.

```
1 // --- CORE FUNCTIONS ---
2
3 // Function to register a newly manufactured part
4 function manufacturePart(uint256 _serialNumber, string memory
   _partName) public {
5     // Security check: ensure serial number isn't already
       used
6     require(parts[_serialNumber].exists == false, "Error:
       Serial Number already registered!");
7
8     // Create the part in storage
9     parts[_serialNumber] = Part({
10         serialNumber: _serialNumber,
11         partName: _partName,
12         manufacturer: msg.sender,
13         currentOwner: msg.sender, // Manufacturer owns it
           initially
14         status: PartStatus.Manufactured,
15         exists: true
16     });
17 }
```



```
18
19 // Function to transfer the part to a new owner (e.g., an
    // Airline)
20 function transferPart(uint256 _serialNumber, address
    _newOwner)
21     public
22     partExists(_serialNumber)
23     onlyPartOwner(_serialNumber)
24 {
25     require(_newOwner != address(0), "Error: Cannot transfer
        to zero address.");
26
27     // Update the owner
28     parts[_serialNumber].currentOwner = _newOwner;
29     parts[_serialNumber].status = PartStatus.InTransit;
30 }
```

Listing 3: Core Business Logic

**Analysis:** Look at the `transferPart` function. It has no `require` statements regarding ownership inside the function body. Why? Because the `onlyPartOwner` modifier handles it automatically! This is clean, modular engineering.

## 5 Step 4: Traceability and Maintenance Logging using Events

In dependability engineering, tracking the history of an item is as important as its current state. A Turbine Blade might undergo maintenance, inspections, or repairs.

Storing a massive array of strings on the blockchain for every maintenance note is extremely expensive in terms of Gas. Instead, we use **Events**. Events write data to the Ethereum transaction logs, which are searchable by frontend applications but cannot be accessed by smart contracts, making them very cheap.

### 5.1 Defining Events

We need to announce to the world whenever a part is created, transferred, or undergoes maintenance.

#### Action: Add Events and Maintenance Function

Add the event declarations at the top of your contract (below the struct), and add the maintenance function at the bottom.

```
1 // --- EVENTS ---
2 event PartManufactured(uint256 indexed serialNumber, string
   partName, address manufacturer);
3 event OwnershipTransferred(uint256 indexed serialNumber,
   address oldOwner, address newOwner);
4 event MaintenanceLogged(uint256 indexed serialNumber, address
   mechanic, string report);
5
6 // Update your manufacturePart function to include:
7 // emit PartManufactured(_serialNumber, _partName, msg.sender
   );
8
9 // Update your transferPart function to include:
10 // emit OwnershipTransferred(_serialNumber, msg.sender,
   _newOwner);
11
12 // --- MAINTENANCE FUNCTION ---
13 function logMaintenance(uint256 _serialNumber, string memory
   _report)
14     public
15     partExists(_serialNumber)
16     onlyPartOwner(_serialNumber)
```

```
17     {
18         // Emit an event containing the maintenance report
19         // This permanently stamps the report on the blockchain
20         // ledger
21         emit MaintenanceLogged(_serialNumber, msg.sender, _report
22                                 );
23
24         // Update status to installed assuming maintenance
25         // implies active use
26         parts[_serialNumber].status = PartStatus.Installed;
27     }
```

Listing 4: Adding Events for Traceability

**The indexed Keyword:** By adding `indexed` to the `serialNumber` in the event, external applications (like an airline’s dashboard) can quickly filter the blockchain logs to find all maintenance reports specifically for serial number 12345.

## 6 The Complete Smart Contract

Here is the fully assembled AeroTrack smart contract. Review it to ensure your code matches.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.19;
3
4 contract AeroTrack {
5
6     enum PartStatus { Manufactured, InTransit, Installed, Retired
7         }
8
9     struct Part {
10         uint256 serialNumber;
11         string partName;
12         address manufacturer;
13         address currentOwner;
14         PartStatus status;
15         bool exists;
16     }
17
18     address public regulatoryAuthority;
19     mapping(uint256 => Part) public parts;
20
21     event PartManufactured(uint256 indexed serialNumber, string
22         partName, address manufacturer);
23     event OwnershipTransferred(uint256 indexed serialNumber,
24         address oldOwner, address newOwner);
25     event MaintenanceLogged(uint256 indexed serialNumber, address
26         mechanic, string report);
27
28     constructor() {
29         regulatoryAuthority = msg.sender;
30     }
31
32     modifier partExists(uint256 _serialNumber) {
33         require(parts[_serialNumber].exists == true, "Part does
34             not exist.");
35         _;
```

```
33     modifier onlyPartOwner(uint256 _serialNumber) {
34         require(parts[_serialNumber].currentOwner == msg.sender,
35             "Not the owner.");
36     }
37
38     function manufacturePart(uint256 _serialNumber, string memory
39         _partName) public {
40         require(parts[_serialNumber].exists == false, "Serial
41             Number taken!");
42
43         parts[_serialNumber] = Part({
44             serialNumber: _serialNumber,
45             partName: _partName,
46             manufacturer: msg.sender,
47             currentOwner: msg.sender,
48             status: PartStatus.Manufactured,
49             exists: true
50         });
51
52         emit PartManufactured(_serialNumber, _partName, msg.
53             sender);
54     }
55
56     function transferPart(uint256 _serialNumber, address
57         _newOwner)
58         public partExists(_serialNumber) onlyPartOwner(
59             _serialNumber)
60     {
61         require(_newOwner != address(0), "Invalid address.");
62
63         address oldOwner = parts[_serialNumber].currentOwner;
64         parts[_serialNumber].currentOwner = _newOwner;
65         parts[_serialNumber].status = PartStatus.InTransit;
66
67         emit OwnershipTransferred(_serialNumber, oldOwner,
68             _newOwner);
69     }
70
71     function logMaintenance(uint256 _serialNumber, string memory
72         _report)
```

```
66     public partExists(_serialNumber) onlyPartOwner(  
67         _serialNumber)  
68     {  
69         emit MaintenanceLogged(_serialNumber, msg.sender, _report  
70             );  
71         parts[_serialNumber].status = PartStatus.Installed;  
72     }  
73 }
```

Listing 5: Complete AeroTrack.sol Code

## 7 Step 5: Testing and Debugging in Remix IDE

Engineering requires rigorous testing. We will now simulate the supply chain of an airplane part using multiple accounts in the Remix IDE.

### 7.1 Deployment

1. Open Remix IDE (<https://remix.ethereum.org/>).
2. Paste your complete code into `AeroTrack.sol`.
3. Go to the **Solidity Compiler** tab. Ensure the version matches <sup>0.8.19</sup>and click **Compile**. *Goto the Dep*
4. Under **Environment**, ensure "Remix VM" is selected. You have 10 test accounts with 100 fake ETH each.
5. Click **Deploy**. Your contract will appear at the bottom under "Deployed Contracts".

### 7.2 Simulating the Supply Chain

**Scenario: Account 1 (Manufacturer) creates a part and sends it to Account 2 (Airline).**

#### Task 1: Manufacture the Part

1. Ensure **Account 1** is selected in the "Account" dropdown.
2. Expand the deployed contract.
3. In the `manufacturePart` input fields, enter:
  - `_serialNumber`: 999
  - `_partName`: "Boeing 737 Landing Gear"
4. Click the orange **transact** button.
5. Check the terminal (bottom right). You should see a green checkmark indicating the `PartManufactured` event was emitted!

#### Task 2: Verify State

1. Enter 999 into the blue **parts** button and click it.
2. You will see the struct data returned. Notice that the `currentOwner` is Account 1's address.

**Task 3: Test Access Control (Hacking Attempt!)**

1. Change your active account in the "Account" dropdown to **Account 3** (The Hacker).
2. Try to call `transferPart`. Input 999 for the serial and Account 3's address for the `_newOwner`.
3. Click `transact`.
4. **Observation:** The transaction FAILS. In the terminal, read the error message: *"Not the owner."* Your modifier successfully protected the industrial supply chain!

**Task 4: Legitimate Transfer and Maintenance**

1. Switch back to **Account 1**.
2. Copy the address of **Account 2** from the dropdown.
3. Call `transferPart` with serial 999 and Account 2's address. It will succeed.
4. Switch to **Account 2** (The Airline).
5. Call `logMaintenance` with serial 999 and the report "10,000 hour inspection passed. No fatigue detected."
6. Check the terminal logs to see your maintenance event permanently etched into the blockchain!



## 8 Conclusion and Future Enhancements

Congratulations! You have successfully bridged the gap between theoretical blockchain engineering and a highly critical industrial application. By implementing **Project Aero-Track**, you have proven how smart contracts can eliminate single points of failure in supply chain tracking, directly enhancing the dependability of aviation operations.

### 8.1 Topics for Further Research

As you progress in your engineering career, consider how you might expand this contract:

- **Decentralized Identifiers (DIDs):** How can we cryptographically prove that the "Manufacturer" address actually belongs to Boeing or Airbus?
- **IoT Oracles:** How could sensors on the physical airplane part automatically call the `logMaintenance` function when they detect stress or wear?
- **Upgradability:** If the FAA updates their regulations, how do we update this smart contract without losing all the historical part data?

*"In engineering, trust is good. Cryptographic verification is better."*